Blockchain and Decentralization (CSC5006)

Exercises of lecture 3 — Consensus, FLP, and Paxos

Outline. These paper-and-pen exercises follow lecture 3. They revisit, in order, the specification of consensus, the FLP impossibility result, Paxos, and why consensus needs a majority of correct processes. Exercise 6 builds Paxos again, out of read-write registers; it is longer, and may be finished at home.

Exercise	Topic	Time
1	Specifying consensus	10'
2	Valency, and where FLP stops	20'
3	Running Paxos	20'
4	Why Paxos is safe	15'
5	A majority is necessary	15'
6	Paxos in shared memory	35'

Model. Unless stated otherwise, we use the model of the lecture: n processes, asynchronous, that fail by crashing; a process that never crashes is *correct*. A process proposes $u \in D$ by calling `propose(u)`, and decides v when the call returns v. Consensus requires:

(validity) if a process decides u, then u was proposed;

(agreement) no two processes decide different values;

(termination) every correct process eventually decides.

Exercises

Exercise 1 — Specifying consensus (10')

Consider the binary case, $D = \{0, 1\}$, and $n \geq 3$ processes, among which one may crash. Each algorithm below is a candidate for consensus.

A1. Every process decides 0, immediately.

A2. Every process decides its own proposal, immediately.

A3. Process p_0 writes its proposal in a shared register R; every process waits until $R \neq$ `null`, then decides R (the algorithm of slide "The easy case: no failures").

A4. Every process sends its proposal to all, waits for the proposals of all the n processes, and decides the smallest one.

A5. Every process sends its proposal to all, waits for the proposals of $n-1$ processes (itself included), and decides the smallest one.

1. For each algorithm, give the property (or properties) of consensus that it violates, with an execution that shows it.

2. Which of these violations are violations of a *safety* property, and which of a *liveness* property? Recall the difference: a safety violation is visible in a finite prefix of the execution; a liveness violation is not.

3. In a blockchain, D is a set of blocks. Explain, with a double spend, why *validity* alone is too weak for a ledger, and which extra condition a block must satisfy before it may be decided.

Exercise 2 — Valency, and where FLP stops (20')

We follow the proof of the lecture: two processes p and q, binary consensus, read-write registers. A configuration is *k-valent* if only k can be decided from it, and *bivalent* if both 0 and 1 can be. Fischer et al. (1985) state the result; the lecture follows the proof of Loui and Abu-Amara (1987).

1. Show that the initial configuration in which both p and q propose 0 is 0-valent.

2. Let p propose 0 and q propose 1. The lecture says that, if p runs solo, it decides 0.

   (a) Why must p decide at all in a solo run, although q is correct in this execution?

   (b) Why must it decide 0, and not 1? (Hint: what can p not tell apart?)

3. In a critical configuration C, step s (by p) leads to a 0-valent configuration, and step s' (by q) to a 1-valent one. The lecture handles the cases (r, w), (w, r), and (w, w) on the same register x.

   (a) One case is missing. Which one, and why is it easy?

   (b) In the case $s = r(x)$ and $s' = w(x)$, the lecture lets q run solo from $C.s.s'$. Why does it matter that the run is *solo*, that is, that p takes no further step?

4. A *compare-and-swap* register X offers, besides read and write, the atomic operation

   $$\mathsf{CAS}(X, a, b): \quad \text{if } X = a \text{ then } X \leftarrow b; \quad \text{return the value of } X \text{ before the operation.}$$

   (a) Write a wait-free consensus algorithm for any number of processes, with a single CAS register.

   (b) This algorithm does not contradict FLP. Which case of the proof breaks when steps may be CAS operations? Explain why.

5. Paxos is deterministic, works in an asynchronous system, and tolerates crashes. Why does it not contradict FLP? Which property does it keep in every execution, and which one only in some?

Exercise 3 — Running Paxos (20')

We use the code of the lecture (Lamport 1998), recalled below.

```
init:  bal ← 0; vbal ← 0; val ← null
     my_suggestion ← null // local
propose(u)
  my_suggestion ← u
phase1A(b)
  pre:  p = Ω, b ≥ 1, my_suggestion ≠ null
  broadcast ⟨1A, b⟩
phase1B
  pre:  recv ⟨1A, b⟩ from q, bal < b
  bal ← b
  send ⟨1B, b, vbal, val⟩ to q
```

```
phase2A(b)
  pre:  recv ⟨1B, b, b', u⟩ from quorum Q
  Γ ← max {b' :  recv ⟨1B, b, b', _⟩} ∪ {0}
  if Γ = 0 then v ← my_suggestion
  else v:  recv ⟨1B, b, Γ, v⟩
  broadcast ⟨2A, b, v⟩
phase2B(b, v)
  pre:  recv ⟨2A, b, v⟩, bal ≤ b
  bal, vbal ← b; val ← v
  broadcast ⟨2B, b, v⟩
decide(v)
  pre:  recv ⟨2B, b, v⟩ from quorum Q
```

There are $n = 5$ processes $p_1, \ldots, p_5$; a quorum is any set of 3 processes. Process p_i only uses the ballots $b \equiv i \pmod 5$: p_1 uses $1, 6, 11, \ldots$, p_2 uses $2, 7, \ldots$, and so on. We say that p *accepts* (b, v) when it executes `phase2B(b, v)`, and that v is *chosen* at ballot b when a quorum accepts (b, v).

A run. The following happens, in this order.

1) p_1 runs phase 1 at ballot 1 with $\{p_1, p_2, p_3\}$, and broadcasts $\langle$2A, 1, $x\rangle$; only p_1 and p_2 receive it (for now).
2) p_3 runs phase 1 at ballot 3 with $\{p_3, p_4, p_5\}$, and broadcasts $\langle$2A, 3, $y\rangle$; only p_3 receives it (for now).
3) p_4 runs phase 1 at ballot 4 with $\{p_2, p_4, p_5\}$, and broadcasts $\langle$2A, 4, $v_4\rangle$; only p_4 and p_5 receive it (for now).

The other messages are still in transit.

1. p_3 took part in phase 1 of ballot 1. Why was it nevertheless free to propose y at ballot 3? Which value v_4 must p_4 propose at ballot 4?

2. Give the triple `(bal, vbal, val)` of every process at the end of the run. Is a value chosen?

3. Which pending messages, if delivered, would make a value chosen? Can y ever be chosen at ballot 3?

4. Before any pending message is delivered, p_2 starts ballot 7, and receives the answers of phase 1 from $\{p_1, p_2, p_3\}$. Which value must it propose? v_4 was accepted by four processes: is the choice of p_2 safe? Why?

5. Same question, but first $\langle$2A, 4, $v_4\rangle$ reaches p_1. Show that, whatever the quorum that answers p_2 in phase 1, p_2 proposes v_4.

Quorums and leaders.

6. With $n = 4$, a student takes the quorums of size 2. Build an execution in which two different values are decided.

7. Paxos uses one quorum per phase. Howard et al. (2016) observe that a phase-1 quorum need only intersect every phase-2 quorum: $|Q_1| + |Q_2| > n$. With $n = 5$ and $|Q_2| = 2$, what is $|Q_1|$? What does this buy when the leader is stable, and what does it cost when the leader changes?

8. Two processes keep starting ballots, p_1 then p_2 then p_1 .... Give a schedule in which no value is ever chosen. Which mechanism of the lecture rules it out? Raft (Ongaro and Ousterhout 2014) uses randomized election timeouts: why do they help?

Exercise 4 — Why Paxos is safe (15')

We prove agreement for the code above, following Lamport (1998). We assume that a leader executes `phase2A(b)` at most once per ballot.

1. Show that for every ballot b, at most one message $\langle$2A, b, $\cdot\rangle$ is sent. Use the precondition `bal < b` of `phase1B`, not the rule $b \equiv i \pmod n$.

2. An optimization: the leader of ballot 1 skips phase 1, and starts directly at phase 2 with $\langle$2A, 1, `my_suggestion`$\rangle$. Explain why phase 1 is useless at ballot 1. Why is this optimization nevertheless unsafe without the rule $b \equiv i \pmod n$? How many message delays does a decision then take, when the leader is correct and runs alone?

3. Show the invariants: `bal` never decreases, `vbal` $\leq$ `bal`, and `vbal` never decreases.

4. Let v be chosen at ballot b. Prove by strong induction on $b' > b$ the property

$$\mathcal{S}(b') : \quad \text{if } \langle 2\text{A}, b', v' \rangle \text{ is sent, then } v' = v.$$

Let Q_1 be the quorum of phase 1 at b' and Q_2 a quorum that accepted (b, v); take $k \in Q_1 \cap Q_2$.

(a) Show that k accepted (b, v) *before* it answered $\langle 1\text{A}, b' \rangle$.

(b) Deduce that $b \leq \Gamma < b'$.

(c) Conclude, with question 1 when $\Gamma = b$, and with the induction hypothesis otherwise.

5. Deduce agreement: two processes never decide different values.

6. Suppose that, after some time, a single correct leader runs ballots, and a quorum of processes is correct. Show that it eventually decides. Where is this argument used in the lecture?

Exercise 5 — A majority is necessary (15')

Paxos tolerates $f < n/2$ crashes, and the lecture states that this is a minimal condition for consensus with message passing. We prove it, with the *partition argument* (Dwork et al. 1988).

The model is the standard one of lecture 2 (crash failures, reliable links), made more favorable: the system is *partially synchronous*. There is a time GST, unknown to the processes, after which every message is delivered within a known bound Δ; before GST, messages may be arbitrarily delayed. We assume that some algorithm solves consensus in this model when up to f processes may crash, with $f \geq n/2$, and we derive a contradiction.

1. Why do we prove the result in a partially synchronous system, and not in an asynchronous one?

2. Split the processes in two groups A and B, with $|A| = \lfloor n/2 \rfloor$ and $|B| = \lceil n/2 \rceil$. Show that $|A| \leq f$ and $|B| \leq f$.

3. Run ρ_A: the processes of A propose 0, and those of B crash at the start. Show that every process of A decides 0, by some time t_A. Define ρ_B and t_B symmetrically, with 1.

4. Run ρ: the processes of A propose 0, those of B propose 1, no process crashes, and $GST > \max(t_A, t_B)$. Every message between A and B is delivered after GST. Show that ρ is a legal run, and that, in ρ, the processes of A decide 0 and those of B decide 1. Conclude.

5. In a *synchronous* system, where every message is delivered within Δ from the start, consensus can be solved for any $f < n$ (e.g., FloodSet, in $f + 1$ rounds). Which step of the argument fails there?

6. The ordering service of Hyperledger Fabric runs Raft. How many crashes does a cluster of 3, 4, or 5 orderers tolerate? Why are such clusters usually of odd size?

Exercise 6 — Paxos in shared memory (35')

The lecture builds Paxos with messages. This exercise builds the same algorithm with read-write registers, step by step, in the model of the FLP proof: processes communicate through a shared memory, crash, and are asynchronous. `X`, `Y`, `D`, `C` are shared registers; variables prefixed with `_` are local.

How to disagree. A student proposes the following algorithm, with a single shared register `D`, initially `null`:

```
function propose(u)
  if (D = null) D ← u
  return D
```

1. Give an execution with two processes, proposing 0 and 1, in which they decide different values. Which line is to blame?

A conflict detector, and weak agreement. To repair it, we first *detect* conflicts (below, left). We then use the detector to build *weak agreement* (below, right), also known as *adopt-commit* (Gafni 1998): a process returns a pair, a flag (ADOPT or COMMIT) and a value. Weak agreement guarantees *validity* (a returned value was proposed), *convergence* (if a process commits u, every process returns $(\cdot, u)$), and *termination* (a process that runs solo commits its proposal).

```
X ← 0, Y ← 0 // shared
_id // process id (local, ≠ 0)

function check()
  X ← _id
  if (Y = 0)
    Y ← 1
    return X = _id
  return false
```

```
CD // conflict detector
D ← null, C ← false // shared
_f ← ADOPT // local

function propose(u)
  if (!CD.check())
    C ← true
  if (D = null) D ← u
  if (!C) _f ← COMMIT
  return (_f, D)
```

Each line accesses at most one shared register, except `if (D = null) D ← u`, which is a read of `D` followed (maybe) by a write; a process may be preempted between the two.

The conflict detector.

2. Give an interleaving of processes 2 and 7 in which *both* calls to `check` return `false`. Is this allowed by the specification ("at most one process returns `true`; a process running solo returns `true`")?

3. Prove that at most one process returns `true`. (Hint: assume that p and q both return `true`, and let q be the one that writes `X` first.)

4. A student swaps the first two lines: the process first tests `Y = 0`, then writes `X`, then sets `Y` and reads `X`. Show that this version is wrong.

Weak agreement.

5. Give an execution with two processes, proposing 1 and 0, in which one process commits and the other adopts. Which value does the second one return?

6. Prove *convergence*: if a process p commits u, every process returns $(\cdot, u)$. Proceed in three steps.

(a) Every process $q \neq p$ writes `C ← true`. Why?

(b) Every such write happens after p reads `C`. Why?

(c) Conclude: which value does q read in `D`, and can any process write `D` after p?

Consensus. We run one weak agreement per *ballot*:

```
AC[] // shared array of weak agreements
function propose(u)
   _b ← 0
   while (true)
      _b++
      (_f, u) ← AC[_b].propose(u)
      if (_f = COMMIT) return u
```

7. Prove: if u is committed at ballot b, then every value committed or adopted at a ballot $b' > b$ is u. Deduce agreement and validity.

8. Build an infinite execution with two correct processes in which no process ever decides. Does it contradict the correctness of the algorithm?

9. We add the leader election Ω of the lecture: only the process that Ω designates runs ballots; the others wait until a decision is written in a register. Explain why every correct process then decides (wait-freedom), and why safety does *not* depend on Ω.

10. How to build Ω? Assume that there are n processes, numbered 1 to n, and that every crash is *initial*: a process that crashes takes no step at all. Implement Ω with an array of shared registers `ALIVE[1..n]`, initially `false`, and prove that it eventually returns the same correct process at every process. Why does your construction fail when a process may crash after its first step?

Back to Paxos.

11. With message passing and $f < n/2$, a read-write register can be emulated by asking a quorum (Attiya et al. 1995), so the algorithm above also runs with messages, but slowly. Match it with Paxos: which part of Paxos detects conflicts? Which part writes and reads `D`? What do the weak agreements `AC[b]` become? Why does a Paxos process need only three variables (`bal`, `vbal`, `val`)?

References

Attiya, H., A. Bar-Noy, and D. Dolev (1995). "Sharing Memory Robustly in Message-Passing Systems". In: *Journal of the ACM*.

Dwork, C., N. Lynch, and L. Stockmeyer (1988). "Consensus in the Presence of Partial Synchrony". In: *Journal of the ACM*.

Fischer, M. J., N. A. Lynch, and M. S. Paterson (1985). "Impossibility of Distributed Consensus with One Faulty Process". In: *Journal of the ACM*.

Gafni, E. (1998). "Round-by-Round Fault Detectors (Extended Abstract)". In: *PODC*.

Howard, H., D. Malkhi, and A. Spiegelman (2016). *Flexible Paxos: Quorum Intersection Revisited*. arXiv:1608.06696.

Lamport, L. (1998). "The Part-Time Parliament". In: *ACM Transactions on Computer Systems*.

Loui, M. C. and H. H. Abu-Amara (1987). "Memory Requirements for Agreement Among Unreliable Asynchronous Processes". In: *Advances in Computing Research*. Vol. 4. JAI Press, pp. 163–183.

Ongaro, D. and J. Ousterhout (2014). "In Search of an Understandable Consensus Algorithm". In: *USENIX ATC*.